

Réalisé par :
MARROT Lucas
DJAMAT Zac
CLARET-LAVAL Kyliann

USSI03 - Projet
CNAM - FIP1
2025 - 2026

Enseignant :
DU MOUZA Cédric

Rapport du projet : **Chess.fr**

Rendu Final - 14/06/2026

Sujet : Conception et Développement d'une Plateforme de Jeu d'Échecs en Ligne

GitHub : <https://github.com/LucasMarrot/chess.fr>

le cnam

Sommaire

1. Cadrage et Organisation.....	1
1.1. Définition précise des besoins du client.....	1
1.2. Spécification des besoins.....	2
1.2.1. Contexte du système.....	2
1.2.2. Acteurs identifiés.....	2
1.2.3. Fonctionnalités principales et Cas d'utilisation initiaux.....	3
1.2.4. Risques critiques.....	3
1.3. Organisation du projet.....	4
1.3.1. Méthodologie et Organisation.....	4
1.3.2. Outils de Suivi et de Planification.....	4
2. Conception et Choix Techniques.....	7
2.1. Spécifications fonctionnelles et techniques.....	7
2.1.1. Diagramme des Cas d'Utilisation.....	7
2.1.2. Diagramme de Classes / Composants.....	9
2.1.3. Diagramme d'État.....	11
2.1.4. Diagramme de Séquence.....	13
2.1.5. Architecture du Système (Frontend & Services).....	15
2.2. Choix technologiques.....	17
2.2.1. État de l'art et analyse des solutions du marché.....	17
2.2.2. Analyse comparative et positionnement cible.....	19
2.2.3. Écosystème Frontend et paradigme multiplateforme.....	22
2.2.4. Architecture Backend et persistance des données.....	22
2.2.5. Moteur de règles et Intelligence Artificielle (IA).....	23
3. Réalisation et Implémentation.....	23
3.1. Modularité et choix d'implémentation.....	23
3.2. Algorithme complexe : Gestion d'un mouvement, promotion et synchronisation temporelle.....	23
3.3. Difficultés de réalisation et bilans.....	30
3.3.1. Le développement : Le mirage de la transition Web vers Mobile.....	30
3.3.2. L'organisation : Surestimation, concurrence de projets et limites de l'Agilité...31	
4. Analyse comparative des plannings prévisionnel et réel (Diagrammes de Gantt).....	32
4.1. Le volet Développement (DEV) : de la linéarité théorique au macro-sprint de crise.....	32
4.2. Le volet Documentaire (DOCS) : l'impact de la concurrence inter-projets.....	34
4.3. Bilan critique de l'écosystème Jira.....	36
5. Conclusion.....	36

MARROT Lucas
DJAMAT Zac
CLARET-LAVAL Kyliann

Premier Rendu
Chess.fr

USSI03 - Projet
CNAM - FIP1
2025 - 2026

1. Cadrage et Organisation

1.1. Définition précise des besoins du client

Dans le cadre de ce projet, l'équipe a adopté une organisation inspirée des méthodes de gestion de projet agiles. Deux membres de l'équipe ont pris le rôle de clients, chargés d'exprimer les besoins fonctionnels et les attentes générales vis-à-vis de l'application à développer. Le troisième membre de l'équipe a assuré le rôle de rédacteur et analyste, dont la mission était de formaliser, structurer et clarifier les demandes exprimées par le client afin d'aboutir à un cahier des besoins exploitable pour le développement.

Le client souhaite la réalisation de **Chess.fr**, une **application de jeu d'échecs en ligne**, accessible à la fois sur **mobile (Android et iOS) et sur navigateur web**. L'objectif principal est de proposer une plateforme permettant aux utilisateurs de jouer aux échecs de manière simple, fluide et intuitive, tout en se rapprochant des fonctionnalités offertes par des services existants comme *Chess.com*. L'application devra s'adresser aussi bien à des joueurs débutants qu'à des joueurs plus expérimentés.

Le client exprime en premier lieu un besoin fort concernant la **gestion des utilisateurs**. L'application devra permettre la création de comptes utilisateurs, l'authentification, ainsi que l'accès à un profil personnel. Chaque utilisateur devra pouvoir consulter ses statistiques de jeu, telles que le nombre de parties jouées, le nombre de victoires, de défaites et de parties nulles, ainsi qu'un classement basé sur un système de type ELO. Le client souhaite également que les utilisateurs puissent consulter l'historique de leurs parties afin de suivre leur progression dans le temps.

Concernant le cœur de l'application, le client souhaite un **système de jeu d'échecs complet**. Les utilisateurs devront pouvoir jouer en ligne contre un ami ou contre d'autres joueurs, mais également jouer hors ligne. Le mode hors ligne devra permettre soit de jouer seul contre une intelligence artificielle, soit de jouer à deux sur le même appareil, avec une rotation du plateau pour faciliter l'utilisation. Le client insiste sur la nécessité de proposer plusieurs modes de jeu et plusieurs cadences de temps (*rapide, blitz, classique*), afin de s'adapter aux préférences des différents profils de joueurs.

Un autre besoin important concerne les **interactions sociales**. Le client souhaite intégrer un système de chat permettant aux joueurs de communiquer pendant une partie, ainsi qu'un système de gestion des amis (envoi et acceptation de demandes d'amis). Un filtre de langage devra être mis en place afin de limiter les propos inappropriés et garantir une expérience respectueuse pour l'ensemble des utilisateurs.

Le client exprime également un besoin **pédagogique**. L'application devra proposer un tutoriel expliquant les règles du jeu d'échecs, afin de permettre aux débutants de prendre en main le jeu facilement. Des fonctionnalités supplémentaires, telles que des puzzles d'échecs ou une

analyse des parties jouées à l'aide d'une intelligence artificielle, sont envisagées comme des évolutions possibles si le temps et les contraintes techniques le permettent.

Enfin, le client demande la mise en place d'un **compte administrateur**, permettant la gestion et la modération de la plateforme. Ce compte devra offrir la possibilité de supprimer des utilisateurs, de les bannir en cas de comportement inapproprié et d'assurer le bon fonctionnement général de l'application.

L'ensemble de ces besoins constitue la base fonctionnelle du projet. Ils seront utilisés comme référence pour définir le périmètre du développement, prioriser les fonctionnalités et distinguer les éléments indispensables des fonctionnalités optionnelles ou évolutives.

1.2. Spécification des besoins

1.2.1. Contexte du système

L'objectif est de concevoir et développer Chess.fr, une application de jeu d'échecs multiplateforme (Web, Android, iOS) fonctionnant en temps réel. Le système doit gérer simultanément des sessions utilisateurs sécurisées, la synchronisation des parties multijoueurs, le calcul des performances (système ELO) et le stockage des historiques. Le projet est organisé de manière incrémentale, en ciblant d'abord le cœur de l'application centré sur le jeu de base et l'authentification, avant d'intégrer des modules sociaux, pédagogiques et avancés.

1.2.2. Acteurs identifiés

- **Le Visiteur** : Utilisateur non authentifié arrivant sur la plateforme. Son accès est restreint. Il ne peut effectuer aucune action et se trouve dans l'obligation de s'identifier ou de créer un compte pour accéder aux services de Chess.fr.
- **Le Joueur** : Utilisateur possédant un compte actif. Il accède à l'intégralité des fonctionnalités applicatives et pas la modération.
- **L'Administrateur** : Personne chargée de la supervision globale de la plateforme, de la gestion des données utilisateurs et de la modération.
- **Le Système** : Entité logique autonome responsable de la mise à jour des statistiques, du recalcul du classement ELO en fin de partie, de l'enregistrement de l'historique et du filtrage automatique du langage dans le chat.

1.2.3. Fonctionnalités principales et Cas d'utilisation initiaux

- **Gestion des comptes :**
 - Créer un compte, s'authentifier et se déconnecter.
 - Consulter son profil, son historique de parties, ses statistiques et son classement ELO.
- **Moteur de Jeu d'échecs :**
 - Jouer en ligne contre un adversaire aléatoire ou un ami.
 - Jouer hors ligne en local à deux sur le même écran avec rotation du plateau, ou contre une Intelligence Artificielle.
 - Choisir sa cadence de jeu (blitz, rapide, classique).
- **Interactions sociales :**
 - Gérer une liste de contacts (envoyer, accepter, refuser une demande d'ami).
 - Discuter via un chat en cours de partie (avec filtrage de langage).
- **Administration et Modération :**
 - Consulter la base d'utilisateurs, bannir un joueur ou supprimer un compte inapproprié.
- **Pédagogie :**
 - Accéder à un tutoriel d'apprentissage, résoudre des puzzles d'échecs et analyser ses parties a posteriori via une IA.

1.2.4. Risques critiques

- **Risques techniques et d'infrastructure :** Synchronisation fiable des parties en ligne et gestion robuste des déconnexions/interruptions en plein match. Les temps de réponse doivent être extrêmement rapides pour les parties en *blitz*.
- **Risques algorithmiques :** Utilisation et configuration correcte du moteur de règles (via la bibliothèque chess.js) pour garantir le respect strict des règles officielles, justesse du calcul mathématique du classement ELO et configuration d'une IA performante (basée sur Stockfish) modulable en difficulté.
- **Risques liés à la sécurité :** Stockage sécurisé des mots de passe, protection des données personnelles des utilisateurs et vérification rigoureuse des identifiants lors de l'établissement des sessions.

1.3. Organisation du projet

1.3.1. Méthodologie et Organisation

Pour le développement du projet Chess.fr, l'équipe a fait le choix d'une méthodologie de travail agile, orientée autour d'un suivi structuré et d'une planification claire. Notre organisation se décompose en deux grands axes complémentaires, permettant de garantir à la fois l'avancement technique et la production documentaire :

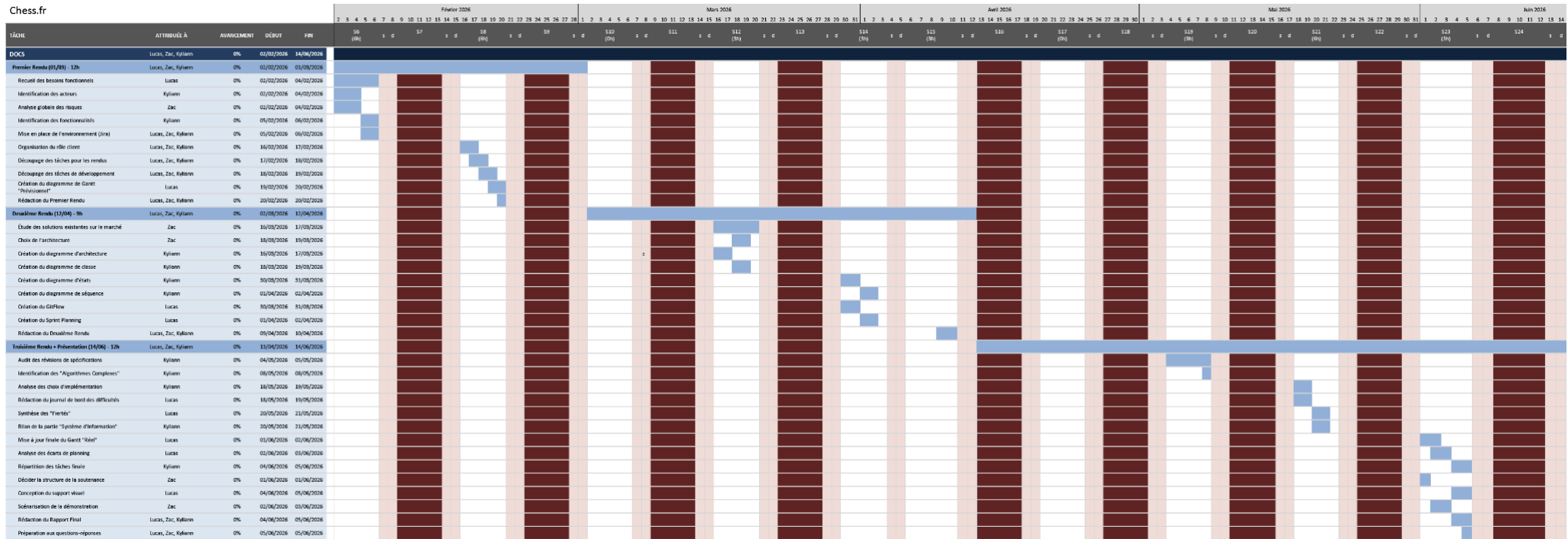
1. **La partie DEV (Développement technique)** : Elle est rythmée par 5 itérations de développement (Sprints), avec un sixième Sprint dit "Bonus" réservé à l'implémentation des fonctionnalités optionnelles et avancées si le temps le permet. Ces Sprints permettent de livrer des incréments fonctionnels réguliers.
2. **La partie DOCS (Documentation et Rapports)** : Elle est structurée autour des 3 phases clés de rendu du projet exigées par l'encadrement (Rendu 1 : Cadrage et Organisation, Rendu 2 : Spécifications et Architecture, Rendu 3 : Bilan de réalisation).

1.3.2. Outils de Suivi et de Planification

Afin de gérer cette organisation, nous avons utilisé deux outils majeurs :

- **Suivi Opérationnel (Jira)** : Pour assurer un suivi rigoureux de l'avancement et une coordination efficace, nous avons mis en place un espace de travail sur Jira. L'arborescence de cet outil reflète fidèlement la structure globale de notre projet, avec la distinction des tâches entre les parties DEV et DOCS et leur répartition au sein des Sprints et des phases de Rendu.
- **Planification et Visualisation (Diagrammes de Gantt)** : Pour visualiser la répartition de notre charge de travail dans le temps et le cadencement des tâches, nous avons élaboré un diagramme de Gantt prévisionnel, segmenté en deux pour une meilleure lisibilité :
 - **Le premier diagramme (DEV)** illustre la progression continue des tâches techniques, calée sur le rythme de nos 5 Sprints (+ Bonus).
 - **Le second diagramme (DOCS)** se concentre sur les livrables documentaires et la préparation des différents rapports exigés pour les trois jalons du projet.

Diagramme de Gantt prévisionnel pour la planification documentaire (Jalons DOCS) :



				Février 2026																												Mars 2026																												Avril 2026																												Mai 2026																												Juin 2026																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																	
				1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8

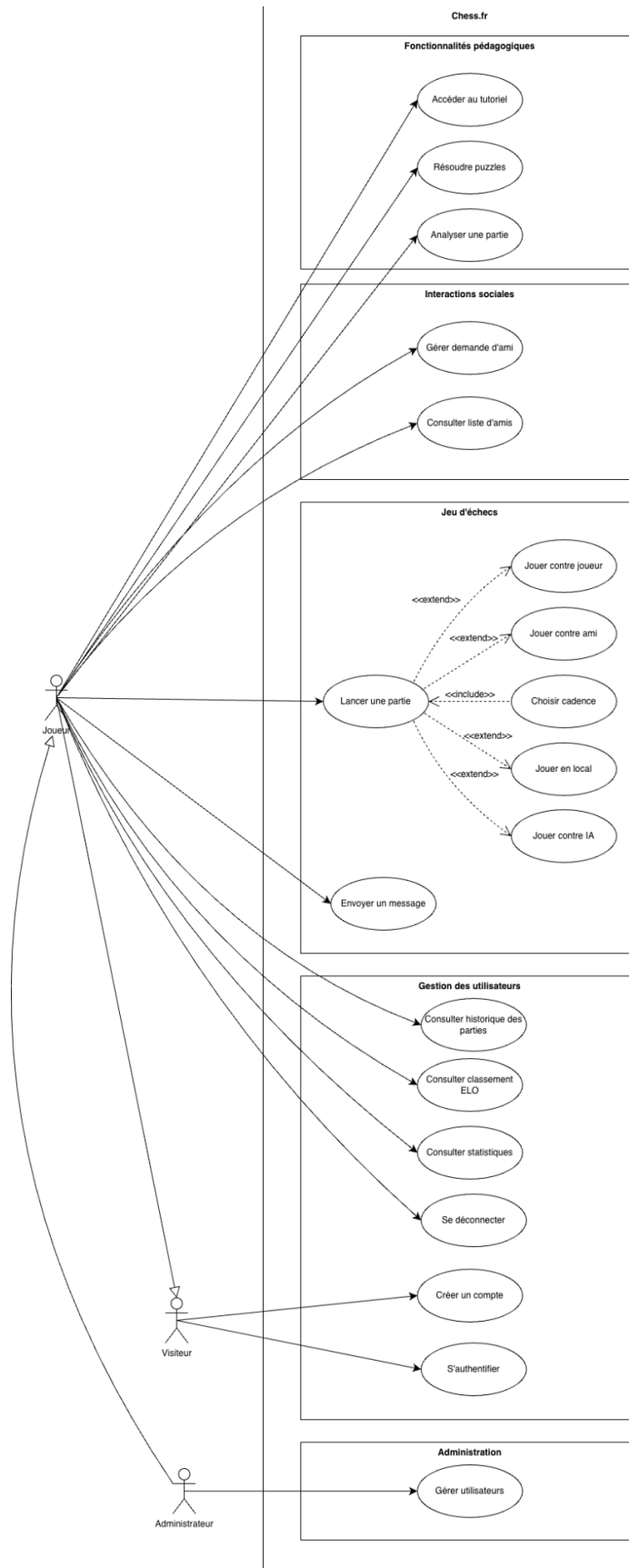
2. Conception et Choix Techniques

2.1. Spécifications fonctionnelles et techniques

Cette section vise à traduire les besoins du client en une modélisation conceptuelle et technique rigoureuse. L'utilisation du langage de modélisation UML (Unified Modeling Language) et de schémas d'architecture spécifiques permet d'offrir différentes vues complémentaires du système Chess.fr : fonctionnelle, structurelle, comportementale et architecturale. Cette démarche de modélisation en amont est cruciale. En effet, elle standardise la communication au sein de l'équipe de développement et garantit que l'implémentation technique répondra fidèlement aux exigences de scalabilité et de maintenabilité du projet.

2.1.1. Diagramme des Cas d'Utilisation

Le diagramme des cas d'utilisation définit le périmètre fonctionnel du système en illustrant les interactions possibles entre les acteurs et l'application. Il permet de cartographier la valeur délivrée à chaque typologie d'utilisateur.



Analyse technique :

Le système identifie clairement trois acteurs avec des niveaux de privilèges et d'accès distincts, illustrant une ségrégation logique des droits :

- **Le Visiteur** : Acteur de base, ses interactions sont intentionnellement restreintes aux cas d'utilisation fondamentaux d'accès au système, à savoir "*Créer un compte*" et "*S'authentifier*". Cette limitation est conçue pour sécuriser la plateforme et inciter à la conversion, obligeant l'utilisateur à s'inscrire pour profiter du service.
- **Le Joueur** : Cet acteur hérite des caractéristiques du visiteur (une fois l'authentification validée) et accède au cœur fonctionnel de l'application. Le diagramme structure ses actions en plusieurs modules logiques distincts, reflétant la richesse de la plateforme :
 - **Gestion des utilisateurs** : Consultation des statistiques globales, suivi de l'évolution du classement mathématique ELO, accès à l'historique détaillé des parties jouées, et déconnexion sécurisée.
 - **Jeu d'échecs** : Le cas d'utilisation central "*Lancer une partie*" possède une relation d'inclusion (<<include>>) stricte vers "*Choisir cadence*". Techniquement, cela signifie que la définition du temps (ex: Blitz 3+2 ou Rapide 10+0) est un paramètre intrinsèque et obligatoire pour initialiser l'échiquier. Il s'étend (<<extend>>) ensuite de manière modulaire vers différents modes de jeu optionnels selon la volonté de l'utilisateur (contre un adversaire humain aléatoire, un ami ciblé, une IA basée sur Stockfish, ou en local sur le même écran).
 - **Interactions sociales & Pédagogie** : Gestion asynchrone des demandes d'amis, communication via le chat in-game, et accès aux modules d'apprentissage (tutoriels et puzzles).
- **L'Administrateur** : Acteur isolé possédant des droits de supervision globale pour le cas "*Gérer utilisateurs*". Cette séparation stricte garantit l'intégrité de la plateforme et permet une modération efficace (bannissements, suppressions) sans interférer avec la logique de jeu classique.

2.1.2. Diagramme de Classes / Composants

Ce diagramme structurel met en évidence l'organisation statique du système. Dans le cadre de notre stack technique orientée composants (React Native), il illustre les relations fondamentales entre les entités du domaine métier, les services d'infrastructure et les composants graphiques de l'interface.

USSI03 - Projet
CNAM - FIP1
2025 - 2026



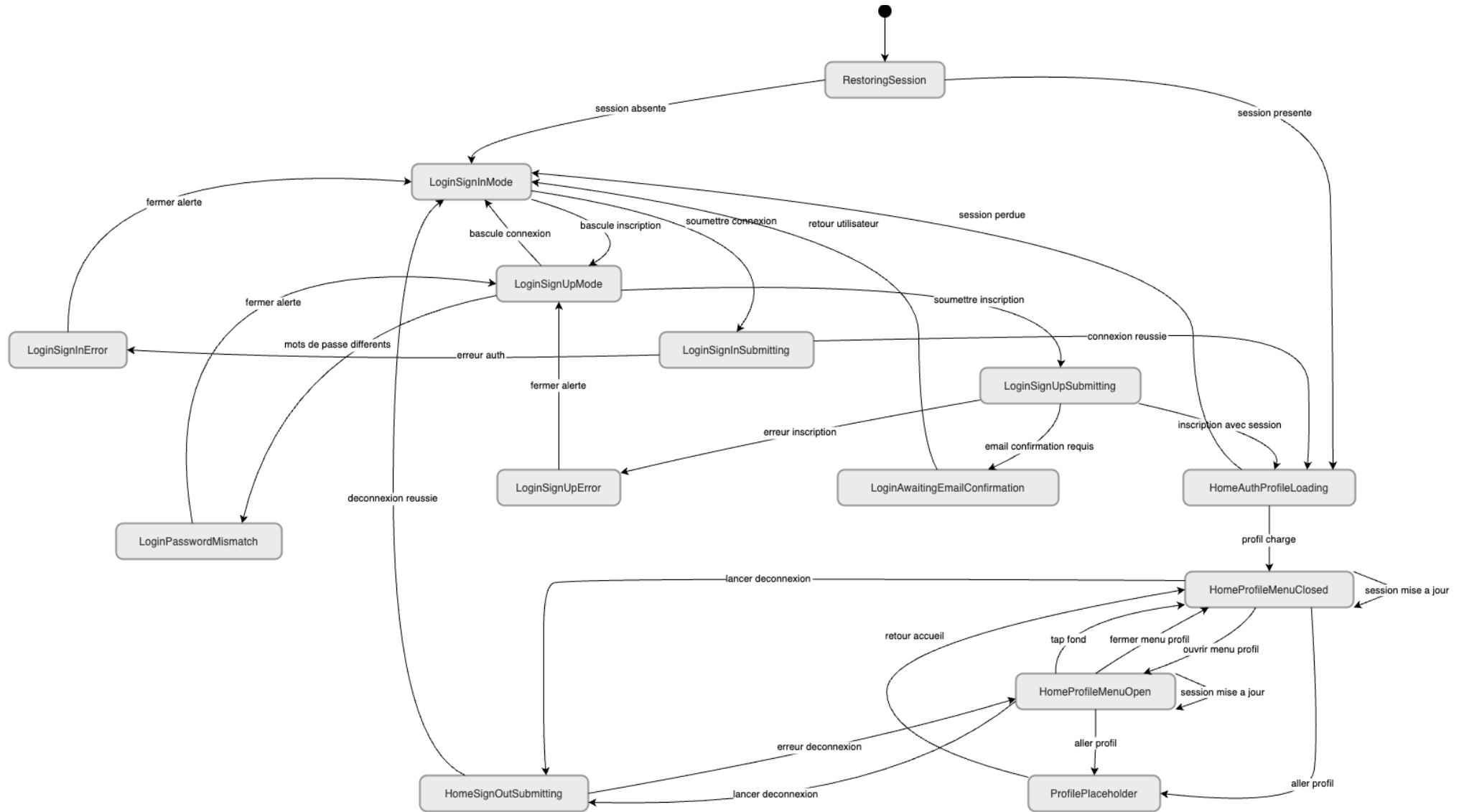
Analyse technique :

Le modèle s'articule autour d'une stricte séparation des préoccupations (SoC - *Separation of Concerns*). Il détaille la nomenclature des classes et leurs cardinalités. On y retrouve les entités fondamentales liées à la logique métier telles que l'**Utilisateur** (porteur du profil et du score ELO), la **Partie** (contenant l'état actuel de l'échiquier sous format FEN), et les **Coups** (enregistrant l'historique des déplacements au format PGN).

Ces entités sont connectées aux couches de persistance et d'affichage. Les dépendances directionnelles démontrent que l'interface graphique est totalement agnostique de la base de données. La communication s'effectue systématiquement au travers d'interfaces de services dédiées (ex: *AuthService* pour la gestion des tokens, *GameService* pour la validation des mouvements). Cette architecture modulaire, basée sur l'injection de dépendances, garantit un très faible couplage. Concrètement, cela permet d'isoler la logique pour réaliser des tests unitaires robustes (en simulant, ou *mockant*, les services) et de modifier le backend (Supabase) sans impacter le code de l'interface visuelle.

2.1.3. Diagramme d'État

Le diagramme d'état (ou machine à états) modélise le cycle de vie dynamique d'un processus spécifique. Il illustre avec précision la complexité technique de la gestion des sessions, des transitions asynchrones et de l'authentification des utilisateurs.



Analyse technique :

Ce diagramme met en exergue la robustesse du flux d'authentification face aux latences réseau et aux comportements imprévus de l'utilisateur.

- **Initialisation fluide** : Le système démarre dans l'état `RestoringSession` pour vérifier silencieusement la présence d'un token JWT valide dans le stockage local de l'appareil. Il bifurque ensuite selon le résultat, réduisant ainsi la friction cognitive pour les utilisateurs récurrents qui sont instantanément connectés.
- **Gestion des modes** : Séparation claire et exclusive entre les états `LoginSignInMode` (formulaire de connexion) et `LoginSignUpMode` (formulaire d'inscription).
- **États transitoires et asynchrones** : L'intégration de l'API Supabase nécessite des états d'attente lors de la soumission réseau. Ces états transitoires, matérialisés par `LoginSignInSubmitting` ou `LoginAwaitingEmailConfirmation`, sont cruciaux pour l'expérience utilisateur : ils permettent de bloquer l'interface (désactivation des boutons, affichage de *spinners*) afin d'éviter les doubles requêtes accidentelles.
- **Gestion proactive des exceptions** : Les états d'erreur (`LoginSignInError`, `LoginPasswordMismatch`) agissent comme des puits de rattrapage sécurisés. Ils permettent de fournir un retour visuel contextuel à l'utilisateur (ex: mot de passe incorrect ou délai d'attente dépassé) et garantissent un retour fluide à la saisie via la transition "*fermer alerte*", sans faire planter l'application.
- **Session active** : Une fois authentifié, le cycle bascule sur les états de l'application principale (`HomeProfileMenuClosed`, `HomeProfileMenuOpen`), avant de se clôturer proprement et de purger les données locales par l'état `HomeSignOutSubmitting` lors de la demande de déconnexion.

2.1.4. Diagramme de Séquence

Le diagramme de séquence offre une vue chronologique des échanges de messages (synchrone et asynchrone) entre les différentes lignes de vie (objets/acteurs) du système pour un scénario donné.



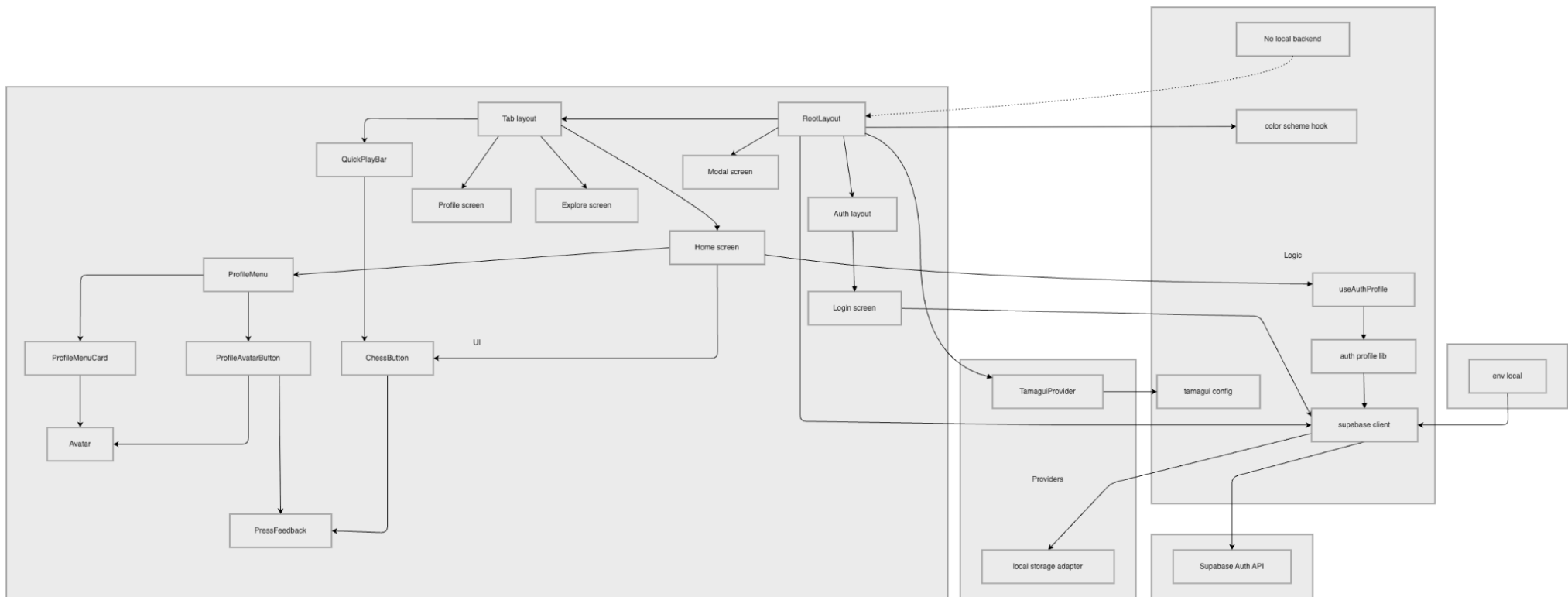
Analyse technique :

Ce diagramme illustre le parcours séquentiel critique, mettant en lumière la nature fondamentalement asynchrone d'une application de jeu en temps réel. Il démontre la verticalité absolue des appels, de l'interaction humaine jusqu'à la base de données :

1. L'action initiée par l'acteur sur le *Frontend* (par exemple, le déplacement du pion e2 en e4) déclenche un appel de fonction, souvent encapsulé et abstrait dans un *hook* React personnalisé.
2. La requête transite par le contrôleur de routing (Expo Router) jusqu'au gestionnaire de service approprié, qui valide d'abord la légalité locale de l'action.
3. Le service émet ensuite une requête asynchrone vers l'API distante (Supabase), souvent sous forme de mutation via un WebSocket pour assurer une diffusion instantanée aux autres joueurs.
4. L'attente de la réponse (résolution de la *Promise* JavaScript) bloque contextuellement l'interface ou affiche un état de chargement/synchronisation pour informer l'utilisateur que le serveur traite sa requête.
5. Le retour de l'API (réussite, échec, ou validation du coup) redescend la chaîne d'appel pour muter l'état global de l'application. Cette mutation d'état déclenche instantanément le re-rendu réactif de l'interface graphique (mise à jour de la position sur l'échiquier virtuel).

2.1.5. Architecture du Système (Frontend & Services)

Ce diagramme architectural fournit une cartographie macroscopique et détaillée de l'écosystème technique déployé côté client. Il lie visuellement la navigation, l'injection de dépendances, le design system et l'accès aux données.



Analyse technique :

L'architecture est savamment segmentée en domaines de responsabilité interconnectés, garantissant une base de code maintenable et évolutive :

1. **Couche de Routage et Layout** : Orchestrée par un système de routage moderne basé sur l'arborescence des fichiers (Expo Router), la hiérarchie démarre au nœud parent `RootLayout`. Elle se scinde intelligemment entre le tunnel public d'authentification (Auth layout, Login screen) et l'application principale protégée (Tab layout, Home screen, Profile screen), facilitant la gestion des accès restreints et la mise en place de *deep links*.
2. **Couche de Présentation (UI)** : L'intégralité de l'arbre des composants React est enveloppée par le `TamaguiProvider`. Ce composant racine est responsable de l'injection globale du *Design System* et de la gestion dynamique du thème (via le *color scheme hook*, permettant de basculer entre mode clair et sombre). Il propage ces tokens de design (couleurs, espacements, typographies) jusqu'aux composants de présentation "feuilles" ou "stupides" (*dumb components*) comme `Avatar`, `ChessButton` ou `PressFeedback`, assurant une cohérence visuelle stricte.
3. **Couche Logique et Accès aux Données** : Le diagramme illustre l'intégration de *hooks* métiers comme `useAuthProfile` qui exploitent la librairie d'authentification. Ces éléments communiquent directement avec le `supabase client`, instancié de manière unique (Singleton) au démarrage de l'application grâce aux clés de configuration sécurisées issues des variables d'environnement (`env local`).
4. **Persistance locale et Résilience** : Le client Supabase est judicieusement couplé à un `local storage adapter` (tel que `AsyncStorage` ou `SecureStore`). Ce mécanisme est vital pour maintenir la persistance de la session active lors d'interruptions de réseau (mode hors ligne partiel) ou lors des redémarrages à froid de l'application, dialoguant de manière transparente et sécurisée avec la Supabase Auth API en arrière-plan.

2.2. Choix technologiques

La conception technique de la plateforme Chess.fr repose sur une analyse préalable et rigoureuse de l'état de l'art des solutions existantes. Cette démarche a permis d'identifier les paradigmes d'architecture performants, d'en cerner les limites, et d'orienter nos choix vers une stack technique moderne, scalable et interopérable.

2.2.1. État de l'art et analyse des solutions du marché

L'écosystème des plateformes d'échecs en ligne est dominé par trois acteurs majeurs. L'étude de ces géants met en lumière les compromis architecturaux auxquels tout service de jeu en temps réel doit faire face.

1. Chess.com (Le leader du marché)

- **Modèle :** Freemium
- **Technologies principales :** Vue.js / TypeScript (Frontend), PHP / Node.js / Go en architecture microservices (Backend), MySQL + Redis (BDD), Stockfish, Cloud GCP/AWS.
- **Fonctionnalités clés :** Jeu en ligne, matchmaking Elo, puzzles, cours, tournois, système social complet.
- **Avantages :**
 - Plateforme extrêmement complète (tout-en-un).
 - Très grande communauté active.
 - Analyse de parties très avancée.
- **Inconvénients :**
 - Interface souvent surchargée (surcharge cognitive).
 - Nombreuses fonctionnalités verrouillées (payantes).
 - Aspect très orienté grand public, manquant d'une touche "premium".

2. Lichess.org (La référence Open Source)

- **Modèle :** 100% Gratuit et Open Source
- **Technologies principales :** Scala 3 / Play Framework (Backend), TypeScript / Snabbdom (Frontend), MongoDB + Redis (BDD), Stockfish.
- **Fonctionnalités clés :** Jeu en ligne (blitz, bullet, classique), puzzles illimités, études collaboratives, API publique.
- **Avantages :**
 - Entièrement gratuit, sans publicité.
 - Performances exceptionnelles (fluidité, ping minimal).
 - Interface simple, épurée et très appréciée des joueurs expérimentés.
- **Inconvénients :**
 - Design très austère, manquant de modernité.

- Moins de contenu pédagogique interactif que ses concurrents.
- Faible gamification pour fidéliser les joueurs occasionnels.

3. Chess24 (L'approche professionnelle)

- **Modèle** : Freemium (orienté contenu premium)
- **Technologies principales** : React (Frontend), Node.js (Backend), CDN vidéo pour le streaming haute disponibilité, Stockfish.
- **Fonctionnalités clés** : Jeu en ligne, cours avec des Grands Maîtres, streaming de tournois, analyse.
- **Avantages** :
 - Contenu pédagogique de très haute qualité (niveau professionnel).
 - Positionnement haut de gamme et sérieux.
- **Inconvénients** :
 - Base de joueurs actifs plus restreinte.
 - Interface moins fluide in-game.
 - Contenu majoritairement payant.

2.2.2. Analyse comparative et positionnement cible

Afin d'évaluer objectivement ces solutions et de définir le positionnement stratégique de Chess.fr, nous avons établi une grille de notation basée sur 5 critères critiques. Chaque critère est évalué sur une échelle de 1 à 5 :

1. **Richesse fonctionnelle** : Évalue la quantité et la diversité des fonctionnalités offertes (modes de jeu exotiques, clubs, variantes).
(1 = MVP basique, 5 = Exhaustivité absolue)
2. **Ergonomie et Design (UX/UI)** : Évalue la clarté, l'esthétique, la modernité et la facilité de navigation de l'interface.
(1 = Confus ou austère, 5 = Premium, intuitif et épuré)
3. **Performance et Réactivité** : Évalue la fluidité in-game, l'optimisation des ressources client et le temps de réponse du serveur.
(1 = Lent/Lourd, 5 = Ultra-rapide et optimisé)

4. **Accessibilité économique** : Évalue la proportion de l'expérience de jeu accessible sans restriction financière.
(1 = Fortement monétisé/Paywall, 5 = 100% gratuit/Open Source)
5. **Accompagnement Pédagogique** : Évalue la qualité et la présence d'outils d'apprentissage (tutoriels, puzzles, analyse IA).
(1 = Inexistant, 5 = Niveau professionnel/Outils de pointe)

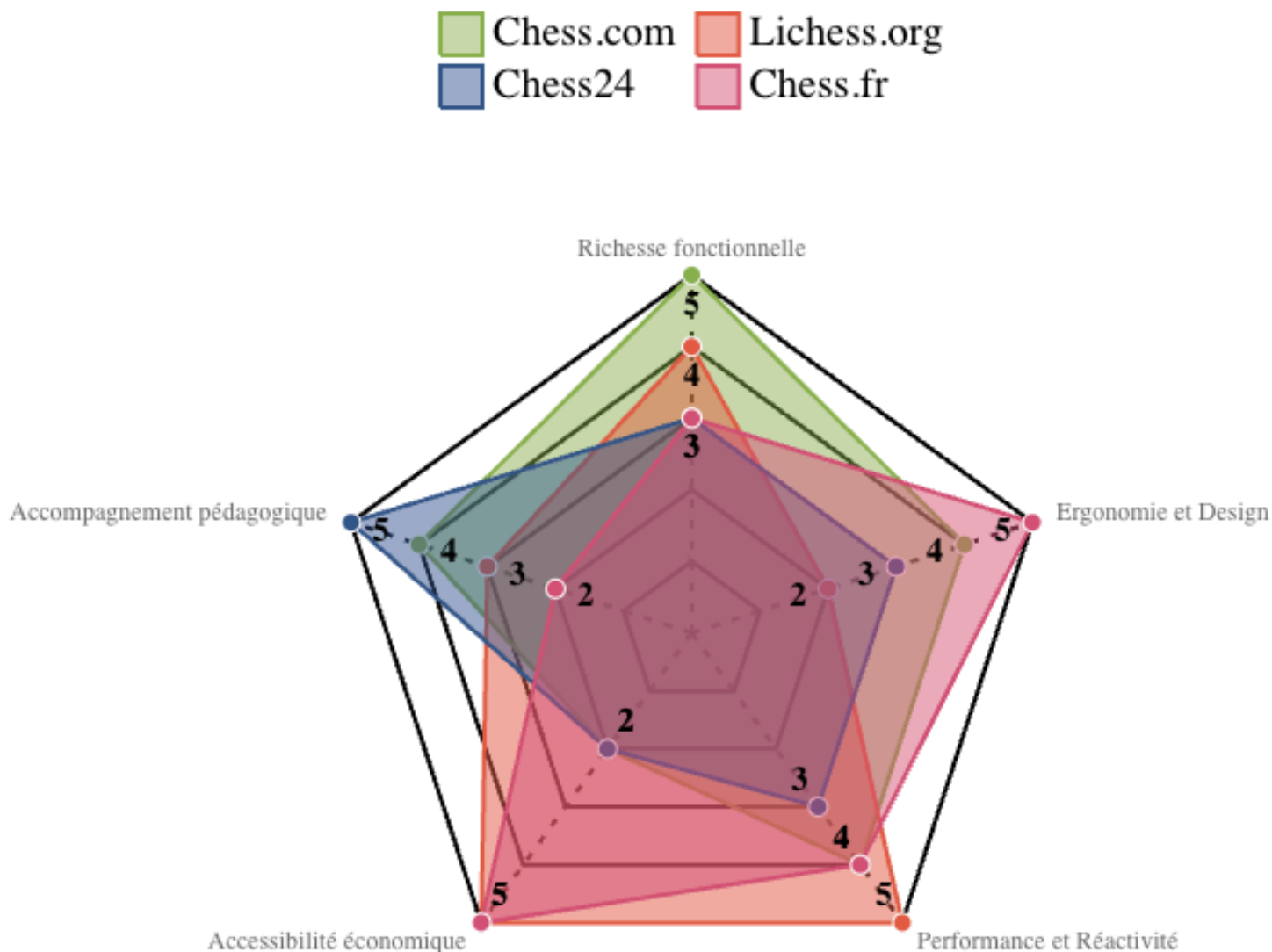
Tableau de synthèse quantitatif

	Chess.com	Lichess.org	Chess24	Objectif Chess.fr
Richesse fonctionnelle	5	4	3	3
Ergonomie et Design	4	2	3	5
Performance et Réactivité	4	5	3	4
Accessibilité économique	2	5	2	5
Accompagnement pédagogique	4	3	5	2
TOTAL	19 / 25	19 / 25	16 / 25	19 / 25

Justification du positionnement :

La représentation de ces données sous forme de graphique radar illustre instantanément la proposition de valeur unique de Chess.fr. L'objectif de notre projet n'est pas de surpasser la concurrence dans une course effrénée aux fonctionnalités ou au contenu pédagogique. Au contraire, nous assumons une approche minimaliste en limitant l'application à l'essentiel.

Notre vision est de proposer un service d'une qualité technique équivalente aux leaders du marché, mais avec une identité radicalement différente : revaloriser la dimension luxueuse, noble et élégante du jeu d'échecs à travers une ergonomie et un design incomparables, tout en garantissant une jouabilité et des performances optimales. Enfin, à l'instar de Lichess, l'ensemble de cette expérience premium sera proposée avec une accessibilité 100% gratuite.



2.2.3. Écosystème Frontend et paradigme multiplateforme

Pour répondre à l'exigence d'une distribution multiplateforme (Web, iOS, Android) stipulée dans le cahier des charges, tout en optimisant le cycle de développement, nous avons fait les choix suivants :

- **React Native avec Expo :**
 - **Mutualisation** : Permet l'implémentation d'une logique métier unique (TypeScript) et la compilation d'interfaces natives pour chaque OS.
 - **Productivité** : Expo facilite drastiquement le déploiement et la gestion complexe des dépendances natives.
- **Tamagui :**
 - **Rôle** : Compilateur d'interface utilisateur (UI) de nouvelle génération.
 - **Justification technique** : Contrairement aux solutions CSS classiques qui calculent les styles à l'exécution, Tamagui analyse l'arbre syntaxique abstrait (AST) pour générer du CSS optimisé à la compilation, garantissant des performances d'affichage quasi-natives.
 - **Design System** : Tamagui agit comme source unique de vérité pour notre design system (couleurs, espacements, tokens), garantissant l'aspect premium et cohérent recherché.

2.2.4. Architecture Backend et persistance des données

Afin d'éviter la maintenance coûteuse d'une infrastructure backend classique pour la gestion des WebSockets, le projet s'appuiera sur **Supabase** (Backend-as-a-Service open source).

- **PostgreSQL et Temps Réel** : Supabase repose sur une base de données relationnelle robuste (PostgreSQL), idéale pour lier les entités du jeu (Utilisateurs, Parties, Historiques). Il expose nativement les mutations de la BDD via des WebSockets sécurisés, ce qui est indispensable pour synchroniser instantanément les coups sur l'échiquier.
- **Authentification centralisée** : La gestion des sessions (JWT, login, logout) sera entièrement déléguée au module Supabase Auth.
- **Sécurité granulaire (RLS)** : La sécurité bénéficie d'une protection au niveau de la donnée elle-même via le *Row Level Security* (RLS). Les règles métier seront appliquées en base de données, empêchant toute manipulation frauduleuse via l'API (ex: interdiction d'insérer un coup à la place de l'adversaire).

2.2.5. Moteur de règles et Intelligence Artificielle (IA)

Développer un moteur d'échecs *from scratch* présente un risque algorithmique critique (gestion des prises en passant, règles des 50 coups, etc.).

- **Logique de jeu avec chess.js** : La validation de la légalité des coups, la gestion de l'état du plateau (format FEN) et l'enregistrement (format PGN) seront délégués à la bibliothèque open source chess.js, garantissant l'exactitude factuelle des règles.
- **Moteur d'analyse avec Stockfish (WASM)** : L'évaluation heuristique des positions et le mode de jeu contre l'IA s'appuieront sur Stockfish compilé en WebAssembly (WASM). Ce choix permet d'exécuter la charge de calcul brutal directement sur le processeur (CPU) du client (mobile/navigateur) via un *Web Worker*, évitant ainsi le gel de l'interface utilisateur tout en économisant les ressources serveur.

3. Réalisation et Implémentation

La phase d'implémentation est l'étape cruciale où les choix architecturaux et fonctionnels définis en amont ont été traduits en code opérationnel. Afin de garantir la maintenabilité et l'évolutivité de la plateforme Chess.fr, nous avons opté pour une séparation stricte des préoccupations (Séparation in Modules), en isolant la logique métier (moteur de jeu, calcul Elo), la gestion d'état global, et les composants d'interface utilisateur (UI).

3.1. Modularité et choix d'implémentation

L'architecture s'appuie fortement sur l'écosystème React Native couplé à Expo. Cette approche déclarative nous a contraints à penser l'application sous forme de composants réutilisables. La logique de persistance des données et de synchronisation temps réel a été encapsulée dans des services dédiés communiquant avec l'API Supabase. Pour la gestion de l'état complexe du jeu en local, nous avons implémenté un gestionnaire d'état global via la librairie Zustand (visible dans notre fichier `use-local-chess-game-store.ts`), qui agit comme un pont entre le moteur de validation des règles (chess.js) et l'interface graphique.

3.2. Algorithme complexe : Gestion d'un mouvement, promotion et synchronisation temporelle

Au cœur du système de jeu en temps réel de Chess.fr se trouve la fonction `attemptMove`. Cet algorithme gère l'état transitionnel lors d'une tentative de déplacement d'une pièce. Sa complexité réside dans l'obligation d'orchestrer, au sein d'un unique flux synchrone, trois couches logiques distinctes : la validation de la légalité mathématique du coup via le moteur

chess.js, l'interception de l'interface pour la gestion asynchrone des promotions, et la mise à jour différentielle des horloges de jeu.

Voici l'implémentation documentée et commentée de cet algorithme clé :

```
attemptMove: (from, to, promotionPiece) => {
  const state = get();
  const { chess, boardOrientation, config, result, clocks } = state;

  // 1. BARRIÈRE DE SÉCURITÉ : Interdiction d'agir si la partie est déjà terminée
  if (result) {
    return { ok: false, requiresPromotion: false };
  }

  // 2. SYNCHRONISATION TEMPORELLE AMONT : Évaluation immédiate du temps restant
  const now = Date.now();
  const elapsedState = getClocksAfterElapsed(clocks, now);

  // Interception si le serveur ou le calcul local détermine une chute d'aiguille (Timeout)
  if (elapsedState.timedOutColor) {
    const timeoutState = applyTimeoutState(
      chess,
      elapsedState.clocks,
      elapsedState.timedOutColor
    );
    set({
      clocks: timeoutState.clocks,
      status: timeoutState.status,
      result: timeoutState.result,
    });
    return { ok: false, requiresPromotion: false };
  }

  // 3. VÉRIFICATION DE LA COHÉRENCE ET DES DROITS DES ACTEURS
  const pieceOnFrom = chess.get(from as ChessJsSquare);
```

```
if (!pieceOnFrom) {
  return { ok: false, requiresPromotion: false };
}

// Empêche un joueur de déplacer une pièce adverse (Contrôle d'intégrité
de la session)
if (pieceOnFrom.color !== chess.turn()) {
  return { ok: false, requiresPromotion: false };
}

// 4. INTERCEPTION DE LA PROMOTION (Rupture du flux linéaire pour L'UI)
const needsPromotion = isPromotionMove(chess, from, to);
if (needsPromotion && !config.autoPromoteToQueen && !promotionPiece) {
  // On fige l'horloge sur l'état écoulé et on bascule L'UI en attente de
choix
  set({
    clocks: elapsedState.clocks,
    pendingPromotion: {
      from,
      to,
      color: pieceOnFrom.color,
    },
  });
  return { ok: false, requiresPromotion: true };
}

// Détermination de la pièce cible de la promotion (par défaut La Reine
ou le choix utilisateur)
const requestedPromotion = config.autoPromoteToQueen
  ? ('q' as PromotionSymbol)
  : toPromotionSymbol(promotionPiece);

try {
  // 5. APPLICATION AU MOTEUR DE RÈGLES : Mutation de l'état interne de
chess.js
  const playedMove = chess.move({
    from,
    to,
```

```
...(requestedPromotion ? { promotion: requestedPromotion } : {}),
});

const nextTurn = chess.turn();
const status = deriveStatusAndResult(chess);
const moveHistory = chess.history();

// 6. CALCUL DYNAMIQUE ET INCRÉMENTATION DE L'HORLOGE (Cadence de type
Fischer)
const isWhiteMove = playedMove.color === 'w';
const incrementedClocks: LocalClockState = {
  ...elapsedState.clocks,
  whiteMs: isWhiteMove
    ? elapsedState.clocks.whiteMs + elapsedState.clocks.incrementMs
    : elapsedState.clocks.whiteMs,
  blackMs: !isWhiteMove
    ? elapsedState.clocks.blackMs + elapsedState.clocks.incrementMs
    : elapsedState.clocks.blackMs,
};

// Attribution de l'horloge active au joueur suivant ou arrêt complet
des timers
const nextClockState: LocalClockState = status.result
  ? {
    ...incrementedClocks,
    activeColor: null,
    started: true,
    lastTickAt: null,
  }
  : {
    ...incrementedClocks,
    activeColor: nextTurn,
    started: true,
    lastTickAt: now,
  };

// Structuration de l'objet de traçabilité historique (PGN / SAN)
const lastMove = {
```

```
    from: playedMove.from as Square,
    to: playedMove.to as Square,
    san: playedMove.san,
    color: playedMove.color,
    isCapture: Boolean(playedMove.captured),
  };

  // 7. ATOMISATION ET MUTATION DE L'ÉTAT DU STORE GLOBAL ZUSTAND
  set({
    fen: chess.fen(),
    turn: nextTurn,
    status: status.status,
    result: status.result,
    moveHistory,
    lastMove,
    clocks: nextClockState,
    // Inversion de la perspective de L'échiquier si L'option auto-flip
    // est active
    boardOrientation: config.autoFlip ? turnToOrientation(nextTurn) :
    boardOrientation,
    pendingPromotion: null,
    ...deriveCheckState(chess),
    ...deriveKingSquares(chess),
  });

  return { ok: true, requiresPromotion: false, move: lastMove };
} catch {
  // En cas d'exception levée par chess.js (Coup illégal, clouage absolu
  // enfreint...)
  return { ok: false, requiresPromotion: false };
}
```

Le cycle de vie d'une tentative de déplacement suit une séquence de sept barrières logiques et mutations critiques :

1. **Validation temporelle amont et barrières d'intégrité :**

Avant toute évaluation géométrique, l'algorithme vérifie que la partie n'est pas déjà terminée en évaluant l'objet `result`. Il procède ensuite immédiatement au recalcul du temps exact écoulé (`getClocksAfterElapsed`) basé sur le timestamp réel du système (`Date.now()`). Si le temps du joueur actif est épuisé (chute d'aiguille), la tentative de mouvement est interceptée, l'horloge est figée et le statut bascule en `timeout`. Une sous-règle mathématique (`hasMatingMaterial`) évalue alors si l'adversaire possède le matériel suffisant pour revendiquer la victoire ou si une nulle par temps dépassé et matériel insuffisant doit être prononcée.

2. Contrôle de cohérence et droits des acteurs :

L'algorithme interroge l'état de l'échiquier pour identifier la pièce située sur la case d'origine (`chess.get(from)`). Il valide l'existence de la pièce et s'assure de l'intégrité des droits de session en vérifiant que la couleur de la pièce correspond strictement au tour actif du système (`pieceOnFrom.color === chess.turn()`), interdisant ainsi toute manipulation frauduleuse ou asynchronisme de saisie.

3. Détection d'interruption par la promotion :

Si le coup implique l'arrivée d'un pion sur sa dernière rangée (`isPromotionMove`), et si l'application n'est pas configurée en mode de sélection automatique ("auto-dame" via `config.autoPromoteToQueen`) ou qu'aucune pièce cible (`promotionPiece`) n'est fournie, le flux d'exécution linéaire est suspendu. L'état global enregistre le mouvement partiel dans l'objet `pendingPromotion` et l'horloge est figée au millième de seconde près afin de passer le relais à l'interface utilisateur pour l'affichage d'un calque de choix.

4. Résolution par le moteur d'échecs :

Dès lors que le coup est standard ou que la pièce de promotion est explicitement confirmée par l'utilisateur, le déplacement est poussé dans l'instance métier `chess.js` via la méthode `chess.move()`. Cette étape délègue la vérification de la légalité absolue du coup (calcul des clouages, validation des trajectoires, prises en passant ou droits au roque) au moteur de règles hors-ligne.

5. Calcul dynamique des horloges :

Le coup étant validé par le moteur, l'algorithme soustrait le temps de réflexion consommé du capital de temps du joueur, puis applique dynamiquement l'incrément défini par la cadence de la partie (cadence de type Fischer, ex: +2 secondes en blitz).

6. Passage de tour et réévaluation des statuts :

Les timers mettent à jour l'attribution de l'horloge active en la basculant sur la couleur du joueur suivant. Simultanément, les fonctions de dérivation (`deriveStatusAndResult`) analysent la nouvelle configuration du plateau pour statuer sur les conditions de fin de partie (Échec et mat, Pat, Nulle par insuffisance de matériel, triple répétition ou règle des 50 coups).

7. Mutation de l'état global et rendu réactif :

En phase terminale, l'algorithme opère une mutation atomique du store Zustand en mettant à jour la chaîne FEN (`chess.fen()`), l'historique complet au format PGN, l'état de mise en échec des rois et l'orientation requise du plateau (`autoFlip`). Ce changement d'état centralisé propage instantanément les modifications vers les composants graphiques pour forcer le re-rendu visuel.

Pour comprendre la robustesse de cette fonction, il convient d'analyser les trois mécanismes d'ingénierie logicielle mis en œuvre :

- **La rupture de flux par la machine à états imbriquée (Promotion asynchrone) :**

Dans une architecture logicielle classique, une action utilisateur déclenche une modification d'état immédiate et unidirectionnelle. Le cas de la promotion introduit une rupture complexe en insérant une machine à états asynchrone au milieu d'une transaction synchrone. En suspendant l'appel à `chess.move` et en stockant temporairement les coordonnées dans l'objet `pendingPromotion`, l'algorithme permet à l'IHM d'afficher un calque (overlay) de sélection de pièce. La boucle d'exécution principale est ainsi mise en attente sans détruire le contexte de la partie ni altérer le calcul temporel sous-jacent.

- **La garantie d'atomicité temporelle contre le "drift" JavaScript :**

L'implémentation d'horloges de jeu en temps réel est une problématique technique critique sous JavaScript. L'utilisation d'un simple `setInterval` asynchrone est proscrite, car le thread unique (Single-Threaded event loop) de JavaScript subit des décalages systématiques (drifts) induits par la charge de calcul des rendus graphiques sur smartphone. Pour contourner cette limite et éliminer tout risque de désynchronisation ou de triche, l'algorithme applique un principe d'atomicité temporelle. En effectuant une soustraction absolue de timestamps (`Date.now() - clocks.lastTickAt`) dès le point d'entrée de la fonction, le temps consommé est calculé indépendamment des performances physiques de l'appareil ou des saccades de l'interface.

- **Indépendance de la couche d’affichage par la sérialisation (FEN/SAN) :**

Afin de respecter le principe de séparation des préoccupations (SoC), l'échiquier graphique de Chess.fr ne maintient aucune logique géométrique ou algorithmique interne. L'intégrité structurelle est portée par la sérialisation des données. À chaque exécution, l'algorithme génère et stocke une chaîne de caractères normalisée au format FEN (`chess.fen()`) ainsi qu'une notation standardisée (SAN) pour l'historique. Les composants d'affichage se comportent ainsi comme des composants purement présentations ("dumb components"). Ils se contentent de lire le store global et de recalculer matriciellement l'emplacement des pièces, immunisant l'application contre les désalignements d'états visuels.

Cet algorithme illustre parfaitement la complexité de lier un moteur mathématique (le jeu d'échecs) avec les contraintes d'une interface homme-machine asynchrone fonctionnant en temps réel.

3.3. Difficultés de réalisation et bilans

La phase d'implémentation ne s'est pas déroulée sans heurts. Nous avons fait face à deux catégories majeures de difficultés qui ont mis à l'épreuve notre résilience et notre capacité d'adaptation : des obstacles techniques inhérents à notre choix de stack, et des failles dans notre méthodologie organisationnelle.

3.3.1. Le développement : Le mirage de la transition Web vers Mobile

Dès la conception, nous avons été séduits par la promesse d'Expo et de React Native : utiliser nos compétences préexistantes en développement Web pour générer simultanément une application Web et une application Mobile. Nous avons largement sous-estimé la complexité de cette transition. En pensant maîtriser la technologie sous-jacente (JavaScript/React), nous avons anticipé un gain de temps massif.

Cependant, le développement mobile natif a révélé des spécificités techniques très chronophages, propres à cet environnement, que les développeurs web n'ont pas l'habitude de gérer :

- **La gestion du clavier virtuel** : Contrairement au web où le navigateur gère le défilement, le clavier natif du téléphone recouvrait nos champs de saisie (notamment sur l'imposante page d'authentification). Il a fallu implémenter des composants spécifiques (`KeyboardAvoidingView`) capricieux selon l'OS (iOS vs Android).
- **Les zones de sécurité (Safe Areas)** : Les téléphones modernes comportent des encoches (notches) et des barres de navigation natives. Nos interfaces web

débordaient sur ces zones, nécessitant un remaniement lourd du layout.

- **Le paradigme de Navigation** : Le routage sur mobile (React Navigation / Expo Router) fonctionne sous forme de "piles" (Stacks) et non de liens URL simples. La gestion des retours en arrière et du cycle de vie des composants (écrans montés/démontés en arrière-plan) a généré de nombreux bugs d'état.
- **Les événements tactiles complexes** : Sur l'échiquier, remplacer le comportement natif de défilement de l'écran par un "Drag & Drop" (glisser-déposer) fluide des pièces s'est avéré extrêmement complexe. Gérer la capture du geste (Gesture Handlers) sans faire "lagger" une grille de 64 éléments rendus simultanément a englouti une part immense de notre temps de développement.

3.3.2. L'organisation : Surestimation, concurrence de projets et limites de l'Agilité

Sur le plan de la gestion de projet, notre familiarité (supposée) avec l'écosystème web nous a rendus excessivement ambitieux. Nous avons dimensionné le projet Chess.fr comme s'il s'agissait d'une simple application web dont nous maîtrisions tous les rouages.

Cette surestimation a été violemment heurtée par l'arrivée simultanée du projet de développement Web. La priorisation du temps (hors heures de cours) est devenue chaotique. Le temps de travail personnel à la maison, initialement prévu pour absorber les imprévus techniques de Chess.fr, a été phagocyté par les exigences de ce second projet.

Parallèlement, notre désir de professionnalisme s'est retourné contre nous. Nous avons mis en place un système de tracking et de création de tâches via Jira. Bien que redoutable pour tracer le Gantt prévisionnel et donner une illusion de contrôle, ce système s'est révélé être une "usine à gaz" inadaptée à notre petite équipe et à notre rythme d'étudiants. Le processus de création, de spécification et de distribution des tickets était excessivement chronophage. De plus, nous avons instauré une mécanique de "libre-service" : chaque membre prenait une tâche au fil de l'eau, sans réelles *deadlines* strictes ni micro-jalons. Ce manque de cadre temporel rigide nous a fortement desservi face à la loi de Parkinson (le travail s'étale pour occuper le temps disponible).

L'effondrement du système de Sprints : En conséquence directe des points évoqués (problèmes techniques mobiles imprévus et gestion du temps défailante), notre planification agile initiale a volé en éclats. Le découpage rigoureux en 6 sprints distincts (5 DEV + 1 Bonus) s'est transformé, dans les faits, en 2 grands et douloureux marathons de développement. La dette technique liée aux bugs mobiles chronophages nous a maintenus dans un état d'urgence permanent (le "rush"). Nous n'avions matériellement plus le temps d'exécuter les cérémonies

indispensables clôturant un sprint (Revue, Rétrospectives). Toute notre énergie était allouée à la correction frénétique de comportements instables et à l'intégration au forceps des fonctionnalités restantes, plutôt qu'à l'amélioration itérative de nos méthodes de travail.

4. Analyse comparative des plannings prévisionnel et réel (Diagrammes de Gantt)

L'analyse croisée de nos plannings prévisionnels et de leur exécution réelle constitue un retour d'expérience indispensable. Elle permet de quantifier le glissement de planning (*schedule slippage*), d'identifier les goulets d'étranglement techniques et de comprendre comment les risques matérialisés ont modifié notre trajectoire initiale.

4.1. Le volet Développement (DEV) : de la linéarité théorique au macro-sprint de crise

Notre planification initiale prévoyait un séquençage idéal de 5 sprints thématiques, complété par un sprint optionnel, pour l'intégration progressive des fonctionnalités de Chess.fr (routage, échiquier, authentification, modération, puzzles).

La réalité du terrain a provoqué un effondrement de ce découpage en **deux grands blocs macro-sprints**. La phase de démarrage (Sprints 1 à 3 du prévisionnel) a été fortement ralentie par la courbe d'apprentissage de l'écosystème Expo / React Native. Bien que nous pensions capitaliser sur nos compétences en développement web pour gagner du temps, l'adaptation au développement mobile natif a révélé des barrières complexes : la gestion asynchrone du clavier virtuel, l'ajustement structurel des zones de sécurité (*Safe Areas*), le routage par piles (*Stack Navigation*) et la capture des événements tactiles pour le glisser-déposer des pièces sur l'échiquier.

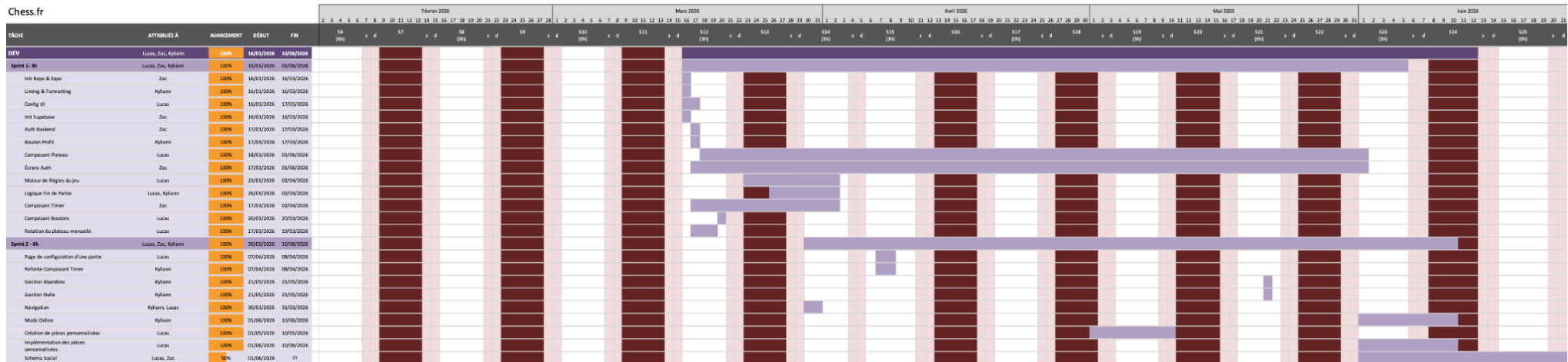
En conséquence, les rituels agiles (revues et rétrospectives de sprints) ont été sacrifiés sous la contrainte de l'urgence. L'équipe est entrée dans un long "tunnel de rush" non structuré (fusionnant les sprints 4, 5 et 6 prévus), où l'implémentation du moteur de règles `chess.js` et de la persistance temps réel Supabase s'est faite en parallèle d'une chasse permanente aux bugs.

MARROT Lucas
DJAMAT Zac
Kyliann

Premier Rendu
Chess.fr

USSI03 - Projet
CNAM - FIP1
2025 - 2026

Diagramme de Gantt réel pour la planification du développement (Sprints DEV) :

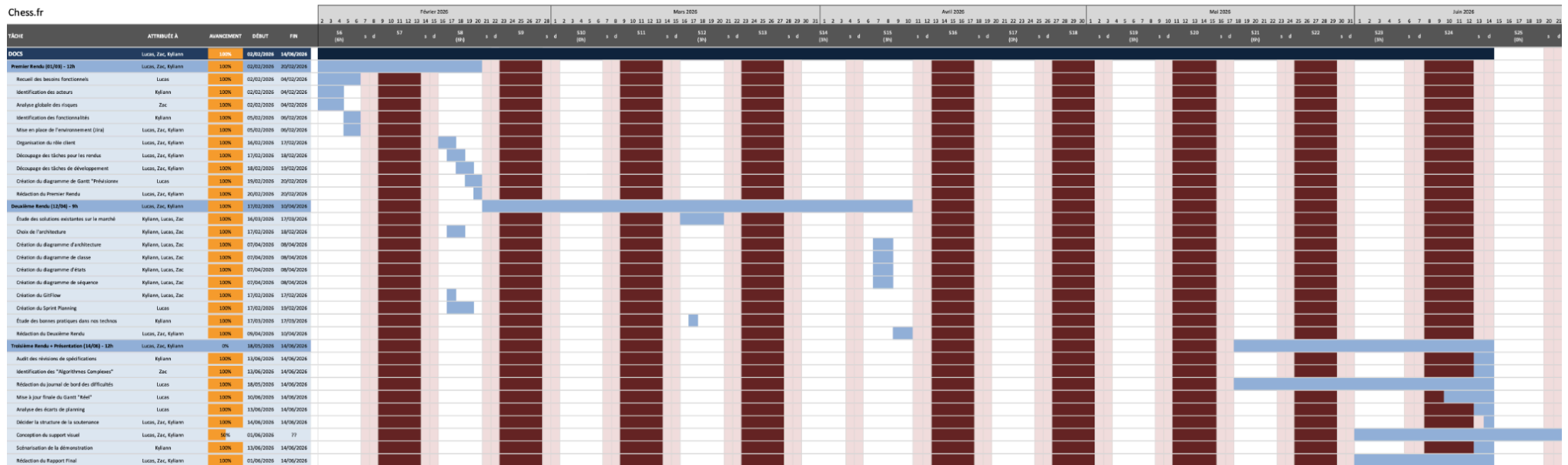


4.2. Le volet Documentaire (DOCS) : l'impact de la concurrence inter-projets

Sur le plan documentaire, le planning prévisionnel tablait sur une production incrémentale fluide, calée sur les trois jalons de rendus officiels.

Le planning réel montre une forte irrégularité induite par des facteurs externes, notamment la collision temporelle avec le projet lourd du cours de développement Web. N'ayant pas défini de deadlines internes strictes pour la rédaction et fonctionnant sur un mode de distribution des tâches en "libre-service" sur Jira, l'équipe a mécaniquement priorisé les urgences du projet web concurrent au détriment de l'avancement régulier des rapports de Chess.fr. Cela a provoqué une charge de travail asymétrique, concentrant la production documentaire dans les derniers jours précédant les rendus obligatoires.

USSI03 - Projet
CNAM - FIP1
2025 - 2026



4.3. Bilan critique de l'écosystème Jira

Si la mise en place d'un espace de travail Jira complet a donné au projet une envergure professionnelle et a facilité la génération de nos diagrammes de Gantt prévisionnels, l'outil s'est avéré contre-productif par rapport à notre vélocité réelle. La sur-spécification et l'administration des tickets ont consommé un temps précieux. De plus, l'absence de dates d'échéance strictes sur les tâches individuelles a favorisé l'étalement du travail (Loi de Parkinson). Pour les futurs projets, un formalisme plus léger (type tableau Kanban épuré avec des *deadlines* hebdomadaires micro-jalonnées) sera privilégié pour éviter cette surcharge de gestion.

5. Conclusion

Le projet Chess.fr arrive à son terme avec un bilan globalement positif, validant les grands axes de notre cahier des charges initial. Malgré les nombreuses turbulences techniques et méthodologiques rencontrées en cours de route, l'équipe a su maintenir son focus sur l'objectif principal : concevoir, développer et déployer une plateforme de jeu d'échecs multiplateforme, robuste et performante.

L'application finale remplit l'ensemble des promesses techniques minimales. Elle offre une expérience fluide en ligne grâce à la synchronisation temps réel des bases de données Supabase, tout en assurant une jouabilité locale complète (mode deux joueurs sur le même écran avec rotation du plateau et intégration stricte des règles via `chess.js`). Avoir abouti à un produit fonctionnel, stable et disponible simultanément sur navigateur web et sur environnement mobile natif constitue une réelle fierté pour l'équipe.

Au-delà du livrable technique, ce projet a été un accélérateur de compétences majeur pour nos futurs profils d'ingénieurs. Sur le plan du développement, nous avons surmonté les pièges du multiplateforme, maîtrisé la gestion d'état global réactif avec Zustand, appréhendé la compilation native avec Expo et configuré une infrastructure asynchrone moderne.

Sur le plan humain et organisationnel, l'expérience s'est révélée tout aussi formatrice. Nous avons mesuré concrètement le danger de surestimer la maîtrise d'une technologie et appris à gérer les priorités face à des charges de travail concurrentes. Ces leçons sur l'importance du cadrage temporel, de la gestion réaliste des risques et de la rigueur des micro-deadlines transforment ce qui aurait pu être de simples erreurs de parcours en un bagage méthodologique précieux pour la suite de notre parcours professionnel.